DATA GOVERNANCE FOUNDATIONS • QUESTION BANK

Questions & Answers

Session 1 — Data Standards, Policies & Documentation
Session 2 — Data Security & Audit Readiness

Databricks & Unity Catalog

Created by Prachi Kabra

Answer follows each question

How to Use This Bank

Every question is followed immediately by a model answer, so this works both as an instructor script and as a revision handout. Pull what fits the moment rather than working through it end to end.

Question types

Type	Purpose	Best used
Recall	Confirm a definition or fact landed	Immediately after teaching
Applied	Make them choose or write something	Mid-segment, keeps attention
Scenario	Judgement under realistic constraints	Discussion, small groups
Discussion	No single right answer	Openers and wrap-ups

Facilitation notes

- Ask, then wait. Count to seven silently before rescuing the room — most groups answer at five.
- For scenario questions, put people in pairs first, then take answers. It doubles participation.
- Score reasoning, not conclusions. An answer naming a Databricks feature without saying who is accountable for it is incomplete.
- Two or three recall questions per segment is plenty — they set pace, they do not assess capability.

> **A note on scope**
> Every answer here stays on the Databricks and Unity Catalog side. Where a question touches regulation, it asks what the platform control is — not for legal interpretation. Point learners at their compliance team for the rest.

Session 1 — Data Standards, Policies & Documentation

1.1 Standards & Policies

Recall

Q1. **What is the difference between a data standard and a data policy? Give one example of each.**

> **A.** A standard is the agreed rule — "monetary values are stored in minor units as BIGINT". A policy is the enforced commitment with consequences — "PII columns must be tagged and masked; violations block deployment". Standards describe, policies compel.

Q2. **Name three things a naming convention should cover in a Unity Catalog environment.**

> **A.** Any three of: catalog naming (environment or domain), schema naming (layer or subject), table naming (snake_case with subject and grain), column naming, boolean prefixes (is_/has_), date and timestamp suffixes (_date/_ts), and key suffixes (_id, _sk).

Q3. **What are the three medallion layers, and what is each responsible for?**

> **A.** Bronze holds raw, as-ingested data. Silver holds cleaned and conformed data. Gold holds business-ready, aggregated data for consumption.

Q4. **What does a three-level name in Unity Catalog look like, and what does each level represent?**

> **A.** catalog.schema.object. Catalog is the top isolation boundary, often environment or domain. Schema groups related objects, often by layer or subject. Object is the table, view or function.

Q5. **Name the four sensitivity tiers in a typical data classification scheme.**

> **A.** Public (no harm if disclosed), Internal (business-only), Confidential (restricted, harmful if leaked), and Restricted/PII (regulated personal or financial data).

Applied

Q6. **You have a column storing money. Write a column name and state its unit convention, then justify both.**

> **A.** amount_cents, stored as BIGINT in minor units. The name carries the unit so no reader has to guess, and integer minor units avoid floating-point rounding errors that corrupt financial aggregation.

Q7. **Given a table of customer support tickets, propose a full three-level name for it and defend each level.**

> **A.** Something like prod.silver.fct_support_ticket. prod isolates the environment, silver signals it is cleaned and conformed rather than raw, and fct_support_ticket names the fact and its grain — one row per ticket.

Q8. **Write the ALTER TABLE statement that tags a column as restricted PII in Unity Catalog.**

> **A.** ALTER TABLE prod.silver.dim_customer ALTER COLUMN customer_email SET TAGS ('data_classification'='restricted', 'pii'='email');

Q9. **A column is named "dt". Rewrite it under a proper convention and explain what information the new name carries.**

> **A.** order_date if it is a date, or order_ts if it carries a time component. The new name states which business event it refers to and, via the suffix, whether it is a date or a timestamp — both of which "dt" leaves ambiguous.

Q10. **Classify these into tiers and justify: product catalog, employee salaries, PAN numbers, internal sales forecast.**

> **A.** Product catalog is Public — no harm if disclosed. Internal sales forecast is Internal — business-only. Employee salaries are Confidential — harmful if leaked, need-to-know plus audit. PAN numbers are Restricted/PII — regulated, requiring tagging, masking and access logging.

Scenario

Q11. **Two teams have built tables for the same business entity with different names and grains. As the steward, what do you do first, and what standard would have prevented it?**

> **A.** First establish which grain each actually represents and who consumes each, then agree a single conformed definition before renaming anything — renaming first breaks consumers. A naming convention plus a published data dictionary with grain stated in the table COMMENT would have surfaced the duplication before the second table was built.

Q12. **A team wants to skip naming conventions on a "temporary" project table that has now existed for a year. How do you respond?**

> **A.** Point out that it has already outlived the exemption — temporary tables that persist are precisely how standards erode. Offer help renaming and documenting it now, and treat "temporary" as requiring an expiry date rather than a permanent waiver.

Q13. **Your organization has no classification scheme and 4,000 tables. Where do you start, and what do you deliberately not do first?**

> **A.** Start with the small set of columns most likely to be regulated — email, phone, government IDs, payment data — and tag those, because they carry the most risk per unit of effort. Do not attempt to classify all 4,000 tables up front; a stalled exhaustive exercise protects nothing.

Q14. **An engineer argues that self-documenting column names make comments unnecessary. Do you agree? Where is the line?**

> **A.** Partly. A good name carries structure and unit — amount_cents tells you the unit. It cannot carry business meaning, source, grain, or edge cases, which is what the comment is for. Names reduce ambiguity; comments explain intent.

1.2 Documentation Practices

Recall

Q1. **Name the three layers of documentation and where each lives in Databricks.**

> **A.** Object-level (what is this table or column) in Unity Catalog comments and tags. Pipeline-level (how is it produced) in DLT/job docs, notebook markdown and Git READMEs. Contract-level (what can I rely on) in data contracts, expectations and SLA notes.

Q2. **What does "documentation as code" mean in practice?**

> **A.** Documentation is stored in Git, reviewed in pull requests, and applied through deployment pipelines — typically as COMMENT clauses embedded in idempotent DDL — so it is versioned alongside the code and cannot drift from what actually ships.

Q3. **What is the "future you" test for a comment?**

> **A.** Write every comment for a competent colleague, or yourself in six months, with zero context. If they would still have to ask "why?", the documentation is not finished.

Q4. **Name three things that belong in a notebook's top markdown cell.**

> **A.** Any three of: purpose, inputs, outputs, owner, and schedule.

Applied

Q5. **Write a CREATE TABLE statement for an orders fact table with a table-level COMMENT and comments on at least three columns.**

> **A.** CREATE TABLE IF NOT EXISTS prod.gold.fct_order_line (order_line_id BIGINT COMMENT 'Surrogate key, one row per line item', order_id BIGINT COMMENT 'FK to fct_order', amount_cents BIGINT COMMENT 'Line revenue in minor units') COMMENT 'Order line grain fact for revenue analytics. Source: silver.orders. Refresh: hourly.';

Q6. **Write the COMMENT ON COLUMN statement for a column holding revenue in minor units, and make the comment pass the "future you" test.**

> **A.** COMMENT ON COLUMN prod.gold.fct_order_line.amount_cents IS 'Line revenue in minor units (paise). Excludes tax and shipping. Sourced from silver.orders.amount; never negative.' — it states unit, scope, source and constraint rather than restating the name.

Q7. **Which TBLPROPERTIES would you set to record quality tier and owner, and why put them there rather than in a wiki?**

> **A.** TBLPROPERTIES ('quality'='gold', 'owner'='revenue_analytics'). They belong on the object because they travel with the table, are queryable, and cannot drift out of sync the way a separate wiki page does.

Q8. **Rewrite this weak comment so it is useful: "customer id column".**

> **A.** Something like 'Natural business key sourced from the CRM system; unique per customer; joins to dim_customer.customer_id'. The original restates the column name and adds nothing a reader did not already have.

Scenario

Q9. **A pipeline's only author has left and nothing is documented. What do you reconstruct first, and which Databricks features help you do it?**

> **A.** Reconstruct the dependency graph first — what feeds this and what depends on it — using Unity Catalog lineage, since that tells you blast radius before you change anything. Then use DESCRIBE EXTENDED and information_schema for structure, job run history for schedule and behaviour, and document as you go so the reconstruction is not lost again.

Q10. **Your team documents at the end of each sprint and it keeps slipping. What process change would you propose?**

> **A.** Move documentation into the definition of done rather than a trailing task: COMMENT clauses embedded in the DDL, reviewed in the same pull request as the code. Work deferred to the end of a sprint is the first thing cut when the sprint runs long.

Q11. **A stakeholder asks why documentation should live in Unity Catalog rather than the team wiki they already use. Make the case in three points.**

> **A.** It sits next to the data, so consumers find it at the moment of use rather than hunting for it. It is queryable, so you can generate a dictionary and measure coverage. And it is applied through deployment, so it cannot silently drift from the actual schema the way a wiki page does.

1.3 Data Dictionaries

Recall

Q1. **Name six attributes that belong in a data dictionary entry.**

> **A.** Any six of: physical name, business definition, data type, nullability, classification, source or lineage, owner or steward, and valid values and rules.

Q2. **Where does a data dictionary physically live in Databricks?**

> **A.** As comments and tags on Unity Catalog objects, exposed through information_schema — so the dictionary is generated on demand from the data itself rather than maintained separately.

Q3. **Which information_schema view exposes column comments?**

> **A.** information_schema.columns, in its comment field. Tags come from information_schema.column_tags.

Q4. **What is the difference between a COMMENT and a TAG on a Unity Catalog object?**

> **A.** A COMMENT is human-readable prose describing meaning. A TAG is a machine-readable key–value label that policies, masks and queries can reference programmatically. Use both — comment for people, tag for enforcement.

Applied

Q5. **Write a query against information_schema.columns that produces a data dictionary for one schema.**

> **A.** SELECT table_name, column_name, full_data_type AS data_type, is_nullable, comment AS business_definition FROM prod.information_schema.columns WHERE table_schema = 'silver' ORDER BY table_name, ordinal_position;

Q6. **Write a query that finds every undocumented column in the silver and gold schemas.**

> **A.** SELECT table_schema, table_name, column_name FROM prod.information_schema.columns WHERE table_schema IN ('silver','gold') AND (comment IS NULL OR TRIM(comment) = '');

Q7. **Write a query that computes documentation coverage as a percentage per schema.**

> **A.** SELECT table_schema, COUNT(*) AS total, COUNT(NULLIF(TRIM(comment),'')) AS documented, ROUND(100.0 * COUNT(NULLIF(TRIM(comment),'')) / COUNT(*), 1) AS pct FROM prod.information_schema.columns WHERE table_schema IN ('silver','gold') GROUP BY table_schema;

Q8. **Write a query that lists all columns tagged with a classification, joining tags to columns.**

> **A.** SELECT table_name, column_name, tag_name, tag_value FROM prod.information_schema.column_tags WHERE schema_name = 'silver' ORDER BY table_name, column_name;

Scenario

Q9. **Leadership wants a data dictionary delivered as a spreadsheet, updated monthly. What do you propose instead, and how do you sell it?**

> **A.** Propose generating it from information_schema on demand, optionally exported to a Delta table or dashboard. Sell it on staleness: a monthly spreadsheet is wrong the day after it is produced, whereas a generated dictionary is always current and costs nothing to refresh. If they want a file, automate the export rather than the authoring.

Q10. **Your documentation coverage is 40% across 800 columns. How do you prioritize, and what would you measure to show progress?**

> **A.** Prioritize gold-layer and certified tables first, then anything classified as sensitive, since those have the most consumers and the most risk. Measure coverage percentage per schema on a scheduled dashboard so the trend is visible and each domain owner can see their own number.

Q11. **AI-generated column descriptions are available. What is your policy for using them, and where do you insist on human review?**

> **A.** Use them as a first draft to lower the cost of documenting a large legacy estate, never as final truth. Insist on steward review before anything is marked certified, and never accept AI-generated classification tags for sensitive data unreviewed — a missed PII tag is a compliance failure, and AI cannot know your business rules, units or edge cases.

1.4 Lineage

Recall

Q1. **What three questions does data lineage answer for a data team?**

> **A.** Where did this number come from, what breaks if I change this column, and which reports are affected by this bad upstream load.

Q2. **What are the three granularities of lineage, and what is each used for?**

> **A.** Table-level for impact analysis and dependency mapping, column-level for root-cause analysis and tracing PII propagation, and notebook or job level for ownership and reproducibility.

Q3. **Does Unity Catalog lineage need to be manually annotated? Explain.**

> **A.** No. Unity Catalog captures lineage automatically for any workload running against it — SQL, Python, Scala, notebooks, jobs and DLT — down to column level. You read it rather than maintain it.

Q4. **Which system tables expose lineage, and what is the difference between them?**

> **A.** system.access.table_lineage records table-to-table relationships; system.access.column_lineage records column-to-column relationships, letting you trace a single field back to its specific source columns.

Applied

Q5. **Write a query that finds everything upstream of a given gold table.**

> **A.** SELECT source_table_full_name, target_table_full_name, event_time FROM system.access.table_lineage WHERE target_table_full_name = 'prod.gold.fct_order_line' ORDER BY event_time DESC;

Q6. **Write a query that traces a single column back to its source columns.**

> **A.** SELECT source_table_full_name, source_column_name, target_column_name FROM system.access.column_lineage WHERE target_table_full_name = 'prod.gold.fct_order_line' AND target_column_name = 'amount_cents';

Q7. **Describe the click path in Catalog Explorer to view column-level lineage for a table.**

> **A.** Open Catalog Explorer, navigate to the table, select the Lineage tab to see upstream and downstream tables, click "See lineage graph" for the interactive diagram, then click a column to trace its column-level sources.

Q8. **How would you combine lineage with PII tags to trace where personal data has propagated?**

> **A.** Start from columns tagged as PII in information_schema.column_tags, then follow column_lineage downstream from each to find every table and report the personal data reaches. Classification identifies what is regulated; lineage shows everywhere it flowed.

Scenario

Q9. **A teammate wants to drop a column in a silver table tomorrow. Walk through exactly what you check first and how.**

> **A.** Query downstream column lineage for that column, or open the Lineage tab in Catalog Explorer, to enumerate every dependent table, dashboard and job. Contact the owners of anything found, agree a deprecation window, and only then drop. Doing this after the fact turns a routine change into an escalation.

Q10. **A gold number is wrong and three pipelines feed it. How does lineage change your debugging approach?**

> **A.** Instead of inspecting all three pipelines, use column-level lineage to identify which source columns actually contribute to that specific field, which usually eliminates most of the search space immediately. You debug the contributing path rather than the whole graph.

Q11. **Lineage shows a dashboard depends on a table nobody claims to own. What do you do?**

> **A.** Treat the missing owner as the finding. Use lineage and job history to identify who created and who consumes it, assign an accountable owner, and record that ownership as a tag on the object so the next person does not repeat the search.

1.5 Metadata

Recall

Q1. **Name the three types of metadata and give an example of each.**

> **A.** Technical — data types, schema, size. Business — column definitions, owner, domain. Operational — freshness, row volume, job run success and cost.

Q2. **Which Databricks sources give you operational metadata?**

> **A.** Unity Catalog system tables (including access and lineage tables and job run timelines), DLT pipeline metrics, and job run history.

Q3. **What does DESCRIBE EXTENDED give you that information_schema does not?**

> **A.** Detailed physical and storage properties for a single object — location, format, table properties and partitioning — whereas information_schema is designed for querying structural metadata across many objects at once.

Applied

Q4. **Write a query that shows when each table in a schema was last altered.**

> **A.** SELECT table_schema, table_name, table_type, created, last_altered FROM prod.information_schema.tables WHERE table_schema = 'gold' ORDER BY last_altered DESC;

Q5. **Write a query that determines data freshness for gold tables.**

> **A.** SELECT target_table_full_name, MAX(event_time) AS last_write FROM system.access.table_lineage WHERE target_table_schema = 'gold' GROUP BY target_table_full_name ORDER BY last_write DESC;

Q6. **Propose three metrics for a governance dashboard and name the metadata type each comes from.**

> **A.** Documentation coverage percentage (business metadata, from column comments), table freshness or hours since last write (operational, from system tables), and count of untagged sensitive columns (a mix of technical structure and business classification).

Session 2 — Data Security & Audit Readiness

2.1 Security Fundamentals

Recall

Q1. **Name the three pillars of the CIA triad and one control that serves each.**

> **A.** Confidentiality — access control, encryption or masking. Integrity — constraints, checksums, audit trails. Availability — backups, replication, disaster recovery.

Q2. **What does "defense in depth" mean, and name three layers of it on Databricks.**

> **A.** Layering controls so a failure in one does not expose the data. Layers include network (private connectivity, IP access lists), identity (SSO, SCIM, MFA), platform (workspace and cluster policies), data (Unity Catalog privileges, masks, row filters), and encryption — all observed by audit logging.

Q3. **In the shared-responsibility model, name two things the platform owns and two you own.**

> **A.** The platform owns physical and host security and default storage encryption. You own the actual grants and group design, and data classification with its masks and policies.

Q4. **Which side of the shared-responsibility line do most real breaches originate from, and why?**

> **A.** The customer side — over-broad grants, unmasked PII, leaked credentials. The infrastructure is hardened by default; the configuration decisions are where judgement, and therefore error, lives.

Scenario

Q5. **A stakeholder says "we are on a managed cloud platform, so security is handled." How do you respond?**

> **A.** Agree on the part that is true — infrastructure, host security and default encryption are handled — then draw the line: who may access which data, how it is classified, and how it is governed remain ours, and that is exactly where most incidents originate. Offer to walk them through the current grants as a concrete illustration.

Q6. **Rank these for a new lakehouse and justify your order: classification, access control, encryption, audit logging.**

> **A.** Encryption is already on by default, so it needs a decision rather than work. Classification comes first of the remaining three, because you cannot protect what you have not identified. Access control follows, since it is the primary confidentiality control. Audit logging is enabled alongside so you can prove the others were in force — a defensible order is more important than a single right answer.

2.2 Encryption

Recall

Q1. **What are the two states of data that encryption protects, and how is each protected on Databricks?**

> **A.** At rest — data on storage, protected by AES-256 on the underlying cloud object storage. In transit — data crossing the network, protected by TLS between the control plane, compute plane and clients.

Q2. **Is encryption at rest something you switch on in Databricks? Explain.**

> **A.** No. It is on by default. What you decide is key custody — platform-managed versus customer-managed keys — not whether encryption happens.

Q3. **What is the difference between platform-managed and customer-managed keys?**

> **A.** Platform-managed keys are generated and rotated by the provider with no effort from you. Customer-managed keys live in your cloud KMS, so you control and can disable them — but you take on rotation and availability responsibility.

Q4. **What is a Databricks secret scope and what problem does it solve?**

> **A.** An encrypted store for credentials, retrieved at runtime with dbutils.secrets.get. It solves credentials being hard-coded into notebooks, config files or Git commits, and secret values are automatically redacted from cell output and logs.

Applied

Q5. **Write the Python that retrieves a database password from a secret scope and uses it in a JDBC read.**

> **A.** password = dbutils.secrets.get(scope="prod-scope", key="db_password") then spark.read.format("jdbc").option("url", jdbc_url).option("user","svc_reader").option("password", password).option("dbtable","public.orders").load()

Q6. **A credential was printed in a notebook cell last month. What is your response, in order?**

> **A.** Treat it as compromised and rotate it immediately — that comes first. Then move it into a secret scope, remove the printing code, and check whether the notebook or its output was committed to Git or exported, since revocation must cover everywhere it travelled.

Q7. **Name three places a credential must never appear, and what to do if it does.**

> **A.** A notebook cell or its output, a config file in a repository, and a Git commit — plus print statements and logs. If a credential appears in any of them, rotate it rather than deleting the line, because deletion does not undo exposure.

Scenario

Q8. **A regulated client asks whether you can render their data unreadable independently of Databricks. What do you offer, and what does it cost you operationally?**

> **A.** Offer customer-managed keys in the cloud KMS, since disabling the key renders the data unreadable without depending on the platform. The cost is that you now own key rotation and key availability — if the key becomes unavailable, so does the data, so it should be adopted deliberately rather than by default.

Q9. **A team proposes storing an API key in a config file in their repo "temporarily". Make the case against it in under a minute.**

> **A.** Once it is committed it is in history forever, visible to everyone with repo access, and cloned onto every developer machine — deleting the line later does not undo any of that. A secret scope takes minutes to set up, keeps the value out of logs automatically, and removes the need to ever rotate under pressure.

2.3 Access Control

Recall

Q1. **State the principle of least privilege in one sentence.**

> **A.** Grant the minimum access required to do the job, and no more.

Q2. **Name the three levels of the Unity Catalog object hierarchy.**

> **A.** Catalog, schema, and table/view/function — with privileges inherited downward and a traversal privilege required at each level above the object.

Q3. **What does USE CATALOG grant, and why is it not enough on its own?**

> **A.** It grants the ability to traverse into a catalog. It is not enough because the user still needs USE SCHEMA on the schema and SELECT on the object itself; traversal alone confers no read access.

Q4. **What is the difference between SELECT and MODIFY?**

> **A.** SELECT allows reading a table or view. MODIFY allows writing — inserting, updating and deleting rows.

Q5. **Why does object ownership matter even when grants look correct?**

> **A.** The owner can always access and re-grant the object regardless of explicit grants, so ownership is an access path that grant reviews miss. Ownership should sit with a group so control is not stranded when an individual leaves.

Applied

Q6. **Write the full set of grants needed for an analyst group to read one gold schema — and explain why each is required.**

> **A.** GRANT USE CATALOG ON CATALOG prod TO `analysts`; GRANT USE SCHEMA ON SCHEMA prod.gold TO `analysts`; GRANT SELECT ON SCHEMA prod.gold TO `analysts`. The first two provide traversal at each level above the objects; the third grants the actual read.

Q7. **Write the statement that shows who currently has access to a given table.**

> **A.** SHOW GRANTS ON TABLE prod.gold.fct_order_line;

Q8. **Write a query against system.information_schema that audits privileges across the silver and gold schemas.**

> **A.** SELECT grantee, privilege_type, table_catalog, table_schema, table_name FROM system.information_schema.table_privileges WHERE table_schema IN ('silver','gold') ORDER BY grantee;

Q9. **A user can see the catalog but gets an error selecting from a table. List the possible causes in order of likelihood.**

> **A.** Missing USE SCHEMA on the schema, then missing SELECT on the table or schema. Less likely: a row filter returning no rows (which reads as empty rather than an error), or the user querying through a compute resource that is not Unity Catalog enabled.

Scenario

Q10. **An analyst asks for ALL PRIVILEGES on a catalog "to move faster". How do you respond and what do you offer instead?**

> **A.** Acknowledge the friction is real, then decline the blanket grant — ALL PRIVILEGES includes write and re-grant rights they do not need and creates an unreviewable access path. Offer SELECT on the specific gold schema they work in, granted to their group so it is inherited automatically, and ask which specific task was blocked so the actual gap can be fixed.

Q11. **An engineer who owned twelve tables has left the company. What is the problem and how do you prevent it recurring?**

> **A.** Ownership is stranded on a disabled identity, so nobody can re-grant or alter those tables without admin intervention. Reassign ownership to a group, and set object ownership to groups as standard so departures become a membership change rather than an access incident.

Q12. **Grants have been made ad hoc to individuals for two years. Design a migration to group-based access without breaking anyone.**

> **A.** Inventory current grants from system.information_schema, cluster users into role-based groups by observed access, then grant the equivalent access to those groups while leaving individual grants in place. Verify each user retains what they need, and only then revoke the individual grants — adding before removing means nobody loses access mid-migration.

2.4 Masking & Row-Level Security

Recall

Q1. **What is the difference between access control and masking?**

> **A.** Access control decides whether you can query a table at all. Masking decides what you see inside it — so many users can share one governed table while sensitive values stay hidden from those who do not need them.

Q2. **Name four masking techniques and when each is appropriate.**

> **A.** Column mask — transform a value on read, e.g. show only the last four digits. Row filter — hide entire rows from some users. Redaction — replace with a constant. Tokenization — swap for a reversible token when the value must be re-identifiable elsewhere.

Q3. **What is a row filter and how does it differ from a column mask?**

> **A.** A row filter is a boolean function attached to a table; rows for which it returns false are invisible to that user. A column mask transforms values within rows the user can already see. Filters remove records, masks obscure fields.

Q4. **What function lets a mask branch on the caller's group membership?**

> **A.** is_account_group_member('group_name'), used inside the masking function to return the raw value to privileged readers and a masked value to everyone else.

Applied

Q5. **Write a masking function that shows a full card number to a privileged group and only the last four digits to everyone else.**

> **A.** CREATE OR REPLACE FUNCTION prod.security.mask_card(card STRING) RETURN CASE WHEN is_account_group_member('pii_readers') THEN card ELSE CONCAT('XXXXXXXX', RIGHT(card, 4)) END;

Q6. **Write the ALTER TABLE that attaches that mask to a column.**

> **A.** ALTER TABLE prod.silver.dim_customer ALTER COLUMN card_number SET MASK prod.security.mask_card;

Q7. **Write a row filter that restricts rows to the user's own region, with an override group.**

> **A.** CREATE OR REPLACE FUNCTION prod.security.region_filter(region STRING) RETURN is_account_group_member('all_regions') OR region = current_user_region(); then ALTER TABLE prod.gold.fct_sales SET ROW FILTER prod.security.region_filter ON (region);

Q8. **Explain what a non-privileged user sees when they query a masked column, and what the query plan actually does.**

> **A.** They see the transformed value — for example XXXXXXXX1234 — with no indication of the raw value. The mask is a UDF evaluated at query time as part of the plan, so the underlying data is unchanged on disk and privileged users reading the same table get the raw value from the same query.

Scenario

Q9. **A team maintains three copies of a table — full, masked, and aggregated — for three audiences. What do you propose instead, and what are the benefits?**

> **A.** One governed table with a column mask and a row filter serving all three audiences. Benefits: a smaller attack surface, no divergence between copies, less storage and pipeline maintenance, one place to change a rule, and far less to audit.

Q10. **Marketing needs to count customers by region but must not see individual PII. Design the controls.**

> **A.** Grant SELECT on a gold aggregate that exposes counts by region rather than the underlying customer table. If they need the detail table, apply a column mask on PII fields and give them a group that is not in the privileged readers list. Verify with SHOW GRANTS and by querying as a member of their group.

Q11. **Someone asks whether masking is enough to satisfy a "restrict access to card data" requirement. What else do you check?**

> **A.** Check who is in the privileged group and whether that membership is justified and reviewed; check object ownership, since owners bypass grants; check whether the raw data is reachable through another table, view or upstream copy via lineage; and confirm audit logging captures access so you can demonstrate the control operated.

2.5 Audit Readiness

Recall

Q1. **What three questions must you be able to answer to be "audit-ready"?**

> **A.** Who can access sensitive data, who actually did, and can you prove the controls were in force throughout the period.

Q2. **Which system table records who accessed what?**

> **A.** system.access.audit.

Q3. **Name three columns from the audit table you would use in an access report.**

> **A.** event_time, user_identity.email, and action_name — commonly with request_params.full_name_arg to identify the object.

Q4. **Why is a last-minute audit scramble a symptom rather than a problem?**

> **A.** It indicates the controls are not operationally monitored — if evidence has to be assembled by hand, nobody was watching between audits. The fix is continuous readiness through scheduled queries and dashboards, not a better scramble.

Applied

Q5. **Write a query showing everyone who accessed a sensitive table in the last seven days.**

> **A.** SELECT event_time, user_identity.email AS user, action_name FROM system.access.audit WHERE request_params.full_name_arg = 'prod.silver.dim_customer' AND event_time > current_timestamp() - INTERVAL 7 DAYS ORDER BY event_time DESC;

Q6. **Write a query that surfaces all privilege changes (grants and revokes) for review.**

> **A.** SELECT event_time, user_identity.email AS changed_by, action_name, request_params FROM system.access.audit WHERE action_name IN ('updatePermissions','grant','revoke') ORDER BY event_time DESC;

Q7. **Write a query that lists every column tagged as PII across the estate as classification evidence.**

> **A.** SELECT catalog_name, schema_name, table_name, column_name, tag_value FROM system.information_schema.column_tags WHERE tag_name = 'pii' ORDER BY schema_name, table_name;

Q8. **Name four monitors you would schedule as alerts, and what each would catch.**

> **A.** Over-broad grants (anyone with ALL PRIVILEGES or raw PII access), unmasked sensitive columns (PII-tagged with no mask attached), failed or unusual access attempts, and changes to secret scopes or new service principals. Each catches drift between reviews rather than at audit time.

Scenario

Q9. **An auditor asks: "Show me everyone who could have read customer PII in Q2, and everyone who did." How do you answer each half?**

> **A.** These are different questions with different sources. "Could have" comes from privileges — SHOW GRANTS and system.information_schema privilege views, plus object owners and any group memberships that confer access. "Did" comes from access events in system.access.audit filtered to the period. Answer both explicitly and do not let one stand in for the other.

Q10. **You discover a table has been readable by all users for six months. Walk through your response, in order.**

> **A.** Determine what is actually in the table and whether it is sensitive, since that sets severity. Revoke the over-broad grant. Query system.access.audit to establish who actually read it over the period — exposure is not the same as access. Report per your incident process, then fix the root cause: how the grant was made and why no monitor caught it for six months.

Q11. **Your organization passes audits but only through heroic manual effort each time. What would you change, and what would you measure?**

> **A.** Convert the evidence-gathering queries into scheduled dashboards and alerts so the state is continuously visible rather than reconstructed. Measure over-broad grant count, unmasked sensitive column count, and time since last access review — if those are green continuously, the audit becomes an export rather than a project.

2.6 Compliance

Recall

Q1. **Name five regulations covered in the session and what each protects.**

> **A.** GDPR — personal data of EU residents. HIPAA — health information. PCI-DSS — payment card data. SOX — financial reporting integrity. RBI/DPDP — financial and personal data in India.

Q2. **What is the core platform demand of PCI-DSS? Of SOX?**

> **A.** PCI-DSS demands masking or tokenization of card data with restricted access. SOX demands change control, segregation of duties, and audit trails supporting financial reporting integrity.

Q3. **What does the "right to erasure" require you to be able to do?**

> **A.** Locate an individual's data everywhere it exists and remove it — which means handling not only the current table but historical Delta versions and backups according to your retention policy.

Applied

Q4. **Map each of these Databricks controls to at least one regulatory requirement: column masks, audit logs, secret scopes, lineage, grants.**

> **A.** Column masks — PCI-DSS card data protection and GDPR de-identification. Audit logs — HIPAA and SOX proof of who accessed what. Secret scopes — credential protection across all of them. Lineage plus tags — GDPR data mapping. Grants — restricting access to authorized users under every regime.

Q5. **Write the steps to fulfil an erasure request, including what to do about historical versions.**

> **A.** Use lineage and PII tags to locate every place the subject's data exists, DELETE from the source of truth, allow downstream tables to rebuild, then VACUUM with a retention period consistent with policy so historical versions no longer hold the data. Record the actions taken as evidence.

Q6. **Why is VACUUM relevant to erasure, and what must you check before running it?**

> **A.** Delta keeps historical versions for time travel, so a DELETE alone leaves the data recoverable. VACUUM removes those files. Before running it, confirm the retention period meets both your erasure obligation and any competing recovery or retention requirement, and that no active readers or streams depend on the versions being removed.

Discussion

Q7. **Where does the platform's responsibility for compliance end and the organization's begin?**

> **A.** The platform provides mechanisms — encryption, privilege enforcement, masking, audit capture. The organization decides what is sensitive, who should have access, whether controls are actually applied, and whether they are reviewed. A compliant platform configured carelessly is not a compliant system.

Q8. **Is a control that exists but is never tested a real control? What would testing look like here?**

> **A.** No — an untested control is an assumption. Testing here means querying as a non-privileged user to confirm the mask actually applies, reviewing grants against intent, confirming audit events appear for a known access, and verifying alerts fire on a deliberately induced condition.

Q9. **What is the risk of treating compliance as a checklist rather than a program?**

> **A.** Checklists capture a moment; systems drift. Grants accumulate, new tables arrive unclassified, and people change roles. Without ownership, monitoring and periodic review, you pass the audit and are non-compliant a month later — with documentation asserting otherwise, which is worse than knowing.

Cross-Session Questions

Use these to connect the two sessions — especially as openers in Session 2 or in a review before Session 3.

Synthesis

Q1. **Session 1 made data findable and understood; Session 2 made it safe and provable. Give one concrete example of a control from each that depends on the other.**

> **A.** A column mask (Session 2) depends on the PII classification tag (Session 1) to know what to protect. Conversely, a data dictionary entry (Session 1) is only trustworthy if access controls (Session 2) prevent unauthorized edits to the definition and the data behind it.

Q2. **Why does a masking policy fail in practice if it is not documented?**

> **A.** A rule nobody can find is a landmine: engineers build around it, consumers mistake masked values for real ones, and auditors cannot verify it operated. Every policy needs documented scope, exemptions, and where it is applied.

Q3. **How do classification tags from Session 1 make Session 2's security controls possible?**

> **A.** Tags turn "which columns are sensitive" into machine-readable state. Masks, access reviews, and audit evidence queries all key off the tag rather than a hard-coded column list, so adding a new PII column inherits protection instead of requiring every policy to be edited.

Q4. **How does lineage serve both a documentation purpose and a compliance purpose?**

> **A.** For documentation it answers where a number came from and what breaks if you change it. For compliance it proves where regulated data flows — the data mapping a GDPR request demands — using the same automatically captured graph.

Q5. **You must protect a new PII column end to end. List every step across both sessions, in order.**

> **A.** Name it per convention, document it with a COMMENT, classify and tag it as PII, apply a column mask, grant access only to the appropriate group, set object ownership to a group, verify with SHOW GRANTS and by querying as a non-privileged user, confirm lineage shows where it propagates, and check it appears in the PII-tag and audit evidence queries.

Q6. **Which single Unity Catalog feature appears in the most controls across both sessions, and why does that matter?**

> **A.** Tags. They carry classification, ownership and certification, and are referenced by masks, access reviews, dictionary generation and audit evidence. It matters because a single well-maintained tagging discipline pays off across documentation, security and compliance simultaneously — and a lapse degrades all three at once.

Hands-on integration tasks

Q7. **Create a table, document every column, tag one as PII, mask it, grant read to a group, and produce the audit evidence — in one notebook.**

> **A.** Expected sequence: CREATE TABLE with COMMENT clauses; COMMENT ON COLUMN for each field; ALTER COLUMN SET TAGS for the PII field; CREATE FUNCTION for the mask and ALTER COLUMN SET MASK; the three GRANT statements; then SHOW GRANTS plus a system.access.audit query. A complete answer verifies the mask by querying as a non-privileged user rather than assuming it works.

Q8. **Produce a single report showing, for each PII column: its business definition, its classification, whether it is masked, and who can read it.**

> **A.** Join information_schema.columns (for comment) to column_tags (for classification), and combine with table_privileges for who can read. Whether a mask is attached is visible from the column metadata in Catalog Explorer or DESCRIBE output. The point of the exercise is that all four facts are queryable rather than tracked manually.

Q9. **Given a table you did not create, assess it against both sessions and produce a gap list.**

> **A.** Check: does it follow naming conventions; does it have a table and column comments; is it classified; is ownership set to a group; are grants least-privilege and group-based; are sensitive columns masked; does lineage show its sources; and does audit history show who has been reading it. The gap list is whichever of those is missing.

Quick-Fire Round

Thirty seconds each, one-line answers. Good for restarting attention after a break or closing a session.

Q1. **Three-level name — what are the three levels?**

> **A.** catalog.schema.object.

Q2. **Which command shows who has access to a table?**

> **A.** SHOW GRANTS ON TABLE.

Q3. **COMMENT or TAG: which one is machine-readable for policy?**

> **A.** TAG.

Q4. **Which schema exposes column comments for querying?**

> **A.** information_schema.columns.

Q5. **True or false: Unity Catalog lineage must be manually maintained.**

> **A.** False — it is captured automatically.

Q6. **Name the privilege needed just to traverse into a schema.**

> **A.** USE SCHEMA.

Q7. **Which system table proves who read a table?**

> **A.** system.access.audit.

Q8. **Bronze, silver, gold — which is business-ready?**

> **A.** Gold.

Q9. **Name one thing that must never appear in a notebook cell.**

> **A.** A credential in plain text.

Q10. **Column mask or row filter: which hides entire records?**

> **A.** Row filter.

Q11. **What does VACUUM have to do with the right to erasure?**

> **A.** It removes historical Delta versions so deleted data is no longer recoverable.

Q12. **Encryption at rest on Databricks — on by default, yes or no?**

> **A.** Yes.

Q13. **Name the three metadata types.**

> **A.** Technical, business, operational.

Q14. **What makes a comment pass the "future you" test?**

> **A.** A colleague with no context would not still have to ask "why?".

Q15. **Groups or individuals — which should receive grants?**

> **A.** Groups.

Grading scenario answers

> For scenario questions, score the reasoning rather than the conclusion. A strong answer names the control, says who owns it, and acknowledges a trade-off. An answer that names a Databricks feature without saying who is accountable for it is incomplete.

— End of Question Bank —

Created by Prachi Kabra • Data Governance Foundations